

- Teamwork to categorize Geo's backlog to develop the roadmap
- Good blogpost on how Geo was built
- Helpful and timely responses from FE and QA teams
- Bad
 - Significant miss during the migration that HEAD was not replicated
 - Many manual steps in the geo runbooks
 - QA test failures - work needs to be done to make them more effective
- Try
 - Get back into a rhythm now that GCP migration is concluded
 - Develop a more detailed roadmap and focus milestones to that roadmap
 - Improve Geo's part of the QA test suite
 - Contribute further technical blog posts

Create/Manage (formerly Platform)

- Good
 - Hit sourcing goal
- Bad
 - No one new was hired
 - Platform team being split up into Create and Manage and Manage getting its own Engineering Manager took focus away from planned performance improvements.
- Try
 - Active sourcing

Gitaly

- Good
 - 1.0 shipped, NFS turned off for .com
 - Gitaly very stable after 1.0 launch
 - 1.1 almost complete, code duplication between gitlab-ce and gitaly-ruby is removed
 - Plans in place for object deduplication and HA Gitaly
- Bad
 - Project discipline has atrophied during push to 1.0
 - Still many urgent projects that need to be done "next" with a small team
 - Still being interim managed by Director of Dev Backend
- Try
 - Source even more aggressively for manager and developer hires
 - Shift team rhythms back to being in sync with GitLab releases, focus on establishing normal project discipline rhythms and processes

Gitter

- Good
 - Open-sourced Gitter Android and iOS apps that got some nice praise
 - No <https://gitter.im/> downtime · with many releases
 - Gitter Topics removed from the codebase, <https://gitlab.com/gitlab-org/gitter/webapp/issues/1947>
 - Less complexity
 - Removes our only usage of React

- Solid deprecation plan, blog post, ability to export, complete removal
- Bad
 - PagerDuty alerts are too noisy
- Try
 - Look at PagerDuty alerts and clean up anything that didn't actually matter

Infrastructure

- Good
 - GCP Migration done in August
 - Adding more people to the team - good onboarding in Americas
- Bad
 - DR implications and analysis with Hurricane Florence
 - SRE team in Europe still on 2 person rotation
 - Small Self inflicted incidents happened with GitLab Pages
- Try
 - Focus on DR strategy for Next Quarter OKRs
 - Focus on prevention of incidents with iterations on change control processes
 - Focus hiring in EMEA

Database

- Good
 - Hired 2nd SRE manager with DBRE background
 - No self inflicted DB incidents
- Bad
 - DBRE hiring moving slower than desired
- Try
 - More focus on meeting sourcing numbers for DBRE role.

Ops Backend

- Good
 - Lots of conversations with the team around throughput and the feedback has been positive. We are seeing comments in retrospective around having large MRs and seeing the value of breaking deliverables to smaller MRs that can be done quickly and reviewed within hours instead of days.
 - Hired an Engineering Manager for the Monitor team. Seth joined the team this quarter.
 - The Configure team regained Mayra back full-time, Thong joined the team and Dylan has done a great job in his new engineering manager role.
 - Elliot joined as the Engineering Manager for the Verify and Release teams.
 - Split Verify and Release team members to provide more focus.
 - Ops team pages were updated with more details
 - Added a director readme
- Bad
 - We are still behind on our hiring. Was not able to hire the second Engineering Manager for Release.
 - Was not able to start on the implementation of the dashboard for throughput so we can capture data for the team.
 - Throughput OKR was not defined in a measurable way which made it hard to quantify and

score.

- Try
 - Focus on implementation of Throughput and other engineering metrics
 - Try new sourcing methods and work on compensation issues

Monitoring

- Good
 - Hired 2 developers. While it still misses our goal of hiring 3, we did pretty well.
 - Weren't able to accurately assess how many candidates were sourced for our positions because of the pooled back-end approach.
 - Error budget was preserved at 100%. This may get harder to pull off as we release more features, increase our surface area, and define SLOs for the services we offer.
 - Onboarded a new manager and a new developer during this time.
- Bad
 - Completely missed the deliverable for the admin dashboard.
- Try
 - Determine what SLOs we will be responsible for and implement them.

CI/CD

- Good
 - Communication around planning improved a lot and involved more of the team
 - Overall adoption of Throughput and smaller MRs
 - Started the process of splitting the team into the Verify and Release teams
 - Steve joined as a Backend Developer
 - Matija joined our team (transitioned from another internal team)
- Bad
 - Behind on hiring goals
 - Development of features frequently gets slowed down by uncovering tech debt or dependencies
 - Runner and pipeline features are hard for Support Team to support leading to an increase in the number of support requests for the team
 - Some big MRs which were painful to get merged before feature-freeze
- Try
 - Look for more ways to break MRs into smaller pieces
 - Async retros to allow for greater participation
 - Look for new ways to help us plan our releases to make things more predictable and less distracting

Secure

- Good
 - CodeQuality successfully handover to Verify.
 - We identified friction points in our process, especially with the versioning of our tools. It will be fixed in Q4.
 - Backend and GitLab skills are improving.
 - We have more and more feedback from customers, to help us prioritize features and bugs.
 - Improved a lot communication and planning, by involving BE, FE, and UX earlier in the process.
- Bad

- Technical prerequisites for storing data in DB slipped several times. It's really hard to define a MVP.
- Could not hire anyone during this quarter. The candidates pool is very low, and sourcing is tedious.
- Storing vulnerabilities data in the DB was more complex than expected. It's also coupled with other features from other teams.
- We have too many domains (five) to cover. Even with CodeQuality gone, we still switch context all the time.
- Try
 - We have a new recruiter for the Secure Team, and new requirements for the candidates. We will target pure Ruby On Rails developers for the open positions.
 - We should have a maintainer in the team.
 - Put in place the rotation on our Release Manager. We need more people in the team for that.
 - Reduce our technical debt, as we add more members in the team.

Configure

- Good
 - Managed to have 2 retros across a very wide timezone range [-7 UTC,+13 UTC]
 - Hired a new backend developer (Thong) and a new frontend engineer (Jacques)
 - Rotating daily standup led to people getting to know each other better
 - Got useful feedback about Auto DevOps by working with the support team
 - A lot more collaboration between cross functional teams from discovery through to implementation
 - Managed to ship Auto DevOps enabled by default for all customers finally
 - Our product vision for 2019 looks really exciting and cutting edge
 - Our asynchronous communication has improved in order to face the challenge of our timezone range
 - Made some progress towards RBAC support which is a feature that has been requested for a long time
 - Shipped protected environments which is a first major improvement to support operators
- Bad
 - Our features still have a very difficult set up process for local development
 - We are not prioritizing our user research findings from Auto DevOps users
 - It seems hard to see the impact of our work due to unknown (but seemingly small) user base
 - Issues still seem to be too large and often take a whole month or slightly more to finish
 - Critical features for Auto DevOps have still not been implemented
 - Not enough hires for backend roles leading to not enough capacity to reach our ambitious goals
 - Missed deliverables occur most months
 - QA test coverage for our products has not improved in some time
 - Auto DevOps on by default has received a great deal of negative feedback from customers
 - Still have not managed to enable Auto DevOps by default on gitlab.com due to our CI infrastructure not being scalable enough
 - Still have not managed to ship RBAC support which is critical for adoption of our Kubernetes integration by most organisations

- Communication via multiple channels (slack, MR, issue) for the same topic is difficult to follow
- Try
 - Async retros so that we do not need somebody to stay up very late to attend retro
 - Work more closely with Quality team to improve our test coverage
 - Get closer to our users by regularly working with support and reading zendesk issues from customers
 - Working closely with sourcing to get many candidates that have very specific skills and interest in Ops and Kubernetes to be able to better sell our team
 - Use threaded discussions on GitLab issues to discuss at a higher bandwidth on GitLab
 - Try out Geekbot so we stay in touch even though our rotating standups are only twice a week
 - Beer call once a quarter to get to know each other even better

Quality

- Good
 - Focus and speciality within the group. Dev Stage dedicated test automation counterparts and Developer Productivity functions have booted up.
 - Setup sub team weekly meetings for collaboration.
 - Implemented triage-package to scale out triage issues to all engineering teams.
- Bad
 - No assigned resources for ops stage.
 - Its a challenge to navigate and prioritize all the work have since we work in multiple projects.
 - Did not reach hiring goal.
 - As we are adding more tests, suites are taking longer to run.
 - Accidentally committed to 4 OKRs, rather than 3 :)
- Try
 - Setup project management process and tooling in a common way for all Quality projects.
 - Initiate better long term planning (roadmap) before epic/issue creation.
 - Come up with a simple MVC for test parallelization in a simple MVC manner

Security

- Good
 - All Q3 Goals were met.
 - We hired 5 Security Engineers in Q3.
 - For S1 security vulnerabilities, our MTTR averages under 30 days, below security industry standards.
 - We have delivered many FGUs, which helps to drive overall awareness of security initiatives at GitLab.
- Bad
 - Our MTTR for S2 security vulnerabilities needs improvement next.
 - Both critical and regular security release processes could use more automation.
 - Timezone coverage for Security team is still mostly concentrated in US and EU zones.
 - Q3 goals not fully reflecting the breadth of security domain deliverables.

- Try
 - Hire security engineers in APAC timezone to increase coverage for security incident response.
 - Increase Q4 goals to cover more breadth of Security team initiatives.
 - Use FGUs as a forum to drive accountability throughout GitLab, to improve MTTRs for security vulnerabilities.

Support

- Good
 - Preparation in advance of Summit to ensure positive customer experience while maximizing support team engagement.
 - Positive progress on exposing key Support metrics into Corporate metrics.
 - Global collaboration on staying on top of our hiring plan.
- Bad
 - Length of time to source and screen excellent Support Engineering Manager candidates in APAC.
 - Experience of losing our first voluntary termination.
 - SLA's for self-managed customers continue to be below goal
 - Some high profile customers/prospects had a disruptive experience during the summit
- Try
 - Establishing a more streamlined candidate to hire process.
 - Clarify expectations for each level of Support Engineering and Support Agent job roles to complement career growth.
 - Ramp up sourcing/hiring in APAC
 - Partner with sales to take especially great care of important customers/prospects prior to renewals and new contracts

Support - Self-Managed

- Good
 - New Hires continue to make an impact quickly
 - Senior Engineers are focusing on deep performance issues and surfacing problems (Gitaly/NFS).
- Bad
 - Small Premium customers are starting to generate too many tickets.
 - We haven't leveraged ticket priority as much as we should.
 - Our bootcamps have atrophied
 - Knowledge is getting siloed
- Try
 - Work with Customer Success to improve onboarding for ALL premium customers
 - Shore up our ticket priority workflows
 - Build a process to verify/enhance bootcamps
 - Encourage the team (seniors specifically) to share more in a group setting

Support - Services

- Good
 - Hit stride in hiring: additional headcount matches volume well.
 - Worked cross-team with Security, Accounts and SMB Team on improving process

- Bad
 - Ticket volume for .com customers is at a level that a miss severely affects SLA performance
 - GitHub app stopped upgrading customer instances after a bad version was posted on [version.gitlab.com](#)
- Try
 - Revisiting breach notifications to ensure we aren't missing tickets because of visibility
 - Encouraging agents to do adhoc pairing sessions for learning and reducing the bystander effect

UX

- Good
 - We hired two excellent UX Designers (Amelia) and soon to be announced!
 - Our hiring pipeline is strong with highly qualified candidates.
 - Despite an increasing number of deliverables per milestone and unexpected UX needs for the 2019 vision, we were able to achieve 100% and 90% respectively on our two OKRs.
 - Our department's comradery and ability to remain aligned and connected has not been impacted by the company and department's rapid growth.
 - Designers embedded in cross-functional teams (stable counterparts) has enhanced collaboration and allowed the UX department to dig deeper into existing features.
 - We added a UX Vision to our department handbook, setting the tone and direction for all of our efforts.
- Bad
 - Design pattern library and design system issues take a long time to review and merge.
 - Our old UX guide is still live as not all of it's material have been moved to [design.gitlab](#).
 - Design discussions still feel fragmented across multiple channels (issues, MR, slack, sync calls).
- Try
 - Async retros for the UX department (separate from group retros) to surface shared problems and solutions.
 - Aggressively break down and iterate on design pattern library and design system issues.
 - Archive the old UX guide and make [design.gitlab](#) the SSOT for UX standards and guidelines.
 - Investigate ways to make design discussion a first-class citizen in GitLab.

UX Research

- Good
 - We created 100% of personas that were requested by UX Designers or the Product team.
 - We conducted 62 user interviews which lead to the creation of 6 new personas.
 - Emily von Hoffmann and Andy Volpe supported UX Research considerably by conducting and analysing user interviews. We couldn't have achieved this OKR without their help.
 - Product Marketing were very supportive of our efforts. They actively participated in Key Reviews and helped us shape the personas' format and content.
 - Despite the disappointing low response rate to the survey, the data we collected provided insight into who qualifies as a churned GitLab.com user and how users first interact with GitLab. We also managed to triangulate the data with user interviews and provide Product with a provisional list of pain points for further exploration.
- Bad
 - In order to identify 5 pain points for users who have left GitLab.com, we created a survey

to send to churned users. We distributed the survey to 8000+ churned GitLab.com users whose details were supplied to us by Product. Unfortunately, the survey only received 126 partial responses. Of those responses, 33% of users confirmed that they were in fact still using GitLab. The recipient list was inadequate for the purposes of our research. This isn't something Product could have foreseen.

- Try

- Closer collaboration with the Product team when creating OKRs.

SECTION: company/okrs/2018-q4.md

<p class="h3">CEO: Grow Incremental ACV according to plan. Sales prospects. Marketing achieves the SCLAU plan. Measure pipe-to-spend for all marketing activities.</p>

- VPE

- Support: Improve Support expectations and documentation for Proof of Concept and Trial prospects, including temporary expanded Support focus for top 10 prospects (defined by Sales). 75% completed with segments redefined and workflows updated.

- Amer East: 95% SLA for Premium/Ultimate/GitLab.com customers and 95% CSAT for all customers

- Amer West: 95% SLA for Premium/Ultimate/GitLab.com customers and 95% CSAT for all customers

- APAC: 95% SLA for Premium/Ultimate/GitLab.com customers and 95% CSAT for all customers

- EMEA: 95% SLA for Premium/Ultimate/GitLab.com customers and 95% CSAT for all customers

- Global team achieved 89% (93% achievement to the 95% goal), +5% increase over CYQ3 and trending up

- CMO: Marketing achieves the SCLAU plan. 8 SCLAU per SDR per month, content 2x QoQ, website information architecture helps 50% QoQ.

- Product Marketing: Complete Information Architecture for customer facing web site pages and add 50 product comparison pages.

- Product Marketing: Deliver 15 enablement sessions each to sales and xDR teams - 30 total sessions.

- Product Marketing: Target 6 industry analyst reports to include GitLab coverage.

- MOps: Implement top three personas into our behavioural and demographic scoring model to get more qualified leads to SDRs.

- Online Growth: Use data-driven approach and SEO/PPC/ABM/Paid social channels to drive traffic to high intent assets and increase signups for gated assets trials, webinars, and demos by 25% compared to last quarter.n

- MPM: Support all field events with pre-event targeting, confirmations, sales notifications, and reminders to drive a minimum of 90% utilization per event.

- MPM: Triple our marketable (aka opted-in for communications) database in Salesforce and Marketo by improving opt-in practices, revamping email communication positioning, and implementing strategic opt-in campaign to engage the database.

- MPM: Launch an on-demand product demo and a weekly LIVE product Q&A session, generating 5x higher net new leads than previous weekly Enterprise Demo.

- Content Marketing: Increase sessions on content marketing blog posts 5% MoM.

- October: 55,583 sessions

- November: 58,362 sessions

- December: 61,280 sessions

- CMO: Measure pipe-to-spend for all marketing activities. Content team based on it, link meaningful conversations to pipe, double down on what works.
 - Online Growth: Improve measurement of pipe-to-spend with more granular data. Work to expand reporting beyond initial pipe-to-spend calculation and ensure online actions are measured and reported.
 - Online Growth: Ensure site is properly tagged for attribution and optimized for crawlers and content and assets are aligned to Information Architecture.
 - MOps: Restructure Marketo/Bizible/SFDC to align the campaign & progression statuses across the system to allow for better multi-touch attribution tracking.
 - MOps: Streamline record management processes. Refresh SFDC layouts with unnecessary fields removed, create record views
- VP Alliances: Establish differentiated relationships with Major Clouds. GitLab as a Service Lol on a major cloud, 500+ leads through joint marketing, GTM and Partner agreement in place tihe a large private cloud provider.
- VP Alliances: establish GitLab as DevOps tool for CNCF. Three keynote/speaking engagements at CNCF events and 1000+ leads from Kubernetes partners
- CFO: Use data to drive business decisions
 - Director of Business Operations: Create scope for User Journey that is documented and mapped to key data sources. 50%
 - Director of Business Operations: Top of funnel process and metrics defined and aligned with bottom of funnel. 70%
 - Data & Analytics: Able to define and calculate Customer Count, MRR and ARR by Customer, Churn by Cohort, Reason for Churn 75%- - Some metrics still require review
 - Director of Business Operations: Looker Explores generated for Customer Support, PeopleOps, GitLab.com Event Data, Marketing Data 100%
 - Data & Analytics: Single Data Lake for all raw and transformed company data - migration to Snowflake complete with DevOps workflow in place, GitLab.com production data extracted and modelled 75%
 - Data & Analytics: Configuration and Business processes documented with integrity tests that sync back with upstream source (SFDC), dbt docs deployed 100%
- Product: Listen to and prioritize top feature requests from customers through sales / customer success. 10 features prioritized. => 130% Complete- 13 items prioritized

CEO: Popular next generation product. Finish 2018 vision of entire DevOps lifecycle. GitLab.com is ready for mission critical applications. All installations have a graph of DevOps score vs. releases per year.

- VPE: Make GitLab.com ready for mission critical customer workloads => 70%, meeting our business goals but by no means perfect, lots of automation to do.
 - Frontend:
 - Spend no more than 15 point of our [error budget]: 0/15
 - Convert a Vue app to use GraphQL as a side-by-side feature with user opt-in (100% - Related issues was delivered behind feature flag)
 - Create, Plan:
 - Spend no more than 15 point of our [error budget]: 0/15
 - 3 charts in gitlab-ui (50% - Base Chart Component and Area Chart are in)
 - Distribution, Monitor and Packaging:
 - Spend no more than 15 point of our [error budget]: 0/15
 - Rewrite and integrate 4 CSS components that align with our design system into GitLab (25% - Markdown type styling)

- Dev Backend:
 - Adopt in collaboration with Product a consistent process for prioritizing work across all backend teams (9/13) => 70%. Need product improvements to help close gap.
 - Perform 3 iterations on retrospective process to be more effective for Engineering (3/3)
- => 80%
- Gitaly:
 - Spend no more than 15 point of our [error budget]: 0/15 => 100%
 - Deliver first iteration of Object Deduplication work => 100%
- Gitter:
 - Spend no more than 15 point of our [error budget]: 0/15 => 100%
 - No gitter.im downtime
 - Shut down billing and embeds on apps-xx => 100%
 - Billing shutdown, <https://gitlab.com/gitlab-org/gitter/webapp/issues/1983>
 - Embeds shutdown, <https://gitlab.com/gitlab-org/gitter/webapp/issues/1984>
- Plan:
 - Spend no more than 15 point of our [error budget]: 0/15 (100%)
 - Complete phase 1 of preparedness for Elasticsearch on GitLab.com (50%)
- Create:
 - Spend no more than 15 point of our [error budget]: 0/15 (100%)
- Manage:
 - Spend no more than 15 point of our error budget]: 15/15 [536
- Geo:
 - Spend no more than 15 point of our error budget]: 15/15 [5215 (100%)
 - Complete all issues in Phase 1 of the DR/HA plan working with the Production Team : We supported the Production Team with all questions and issues raised. Work to enable Geo for DR on GitLab.com continues. (100%)
- Distribution:
 - Spend no more than 15 point of our [error budget]: 0/15 => 100%
 - Automate library upgrades across distributed projects => 75%, completed work related to the omnibus-gitlab package but Helm Charts process still needs work.
- Ops Backend:
 - Drive the implementation of a throughput dashboard for each development team and conduct a training for each: 13/13 dashboards, 1/1 practice training, 3/3 group training (100%)
 - Spend no more than 15 point of our [error budget]: 0/15
- Configure:
 - Spend no more than 15 point of our error budget]: 15/15 [566 (100%)
 - Merge smaller changes more frequently: > 10 average MRs merged per week: 8.7 average (87%)
- Secure:
 - Spend no more than 15 point of our [error budget]: 0/15
 - Merge smaller changes more frequently: > 4 average MRs merged per week: 7 average (90 MRs over 12 weeks)(100%)
- Philippe Lafoucrière:
 - Benchmark 3 new security tools for integration in the Security Products: No security tool was benchmarked (0%)
 - One CFP (Call For Proposals) accepted for a Tech conference: "Enable BeyondCorp for your organization with Cloud Identity" at Google Next (100%)
- Monitor:
 - Spend no more than 15 point of our error budget : 0/15

- Merge smaller changes more frequently: > 6 average MRs merged per week : 79% achieved - Actual weekly average 4.75 MRs
- Kamil:
 - Improve scalability: Reduce CI database footprint as part of Ensure CI/CD can scale in future with Use BuildMetadata to store build configuration in JSONB form: Done (100%)
 - Continuous Deployment: Help with building CD features needed to continuously deploy GitLab.com by closing 5 issues related to Object Storage that are either ~"technical debt" or ~"type::bug": 2 out of 5 (40%)
 - CI/CD:
 - Spend no more than 15 point of our error budget]: 30/15 [5375, 564 (0% - Spent double our goal)
 - Merge smaller changes more frequently: >10 average MRs merged per week. (100% - Actual weekly average was 19 MRs)
 - Infrastructure: Make GitLab.com ready for mission critical workloads
 - Spend no more than 15 point of our [error budget]: 6/15
 - Deliver streamlined and cleaned-up Infrastructure Handbook => 80%
 - SAE: Minimize GitLab.com's MTTR
 - Deliver observable availability improvements in database backup/restore/verification and replication => 90%
 - Launch production queue management for changes, incidents, deltas and hotspots => 85%
 - SRE: Maximize GitLab.com's MTBF
 - Deliver phase 1 DR and HA plan for GitLab.com (66% - 6 of 9 issues in <https://gitlab.com/groups/gitlab-com/gl-infra/-/epics/12>)
 - Implement roadmap and milestone planning with tracking of velocity (80%)
 - Andrew: Observable Availability
 - Deliver General Service Metrics and Alerting, Phase 2: error budgets dashboard and attribution, alert actionability metrics, and MTTD metrics => 90%
 - Deliver Distributed Tracing working in GDK=> 90%
 - Deliver DDoS ELK alerting, documentation and training => 80%
 - Delivery: Streamline releases
 - Create release pipelines
 - Create a CD blueprint for 2019
 - Quality:
 - Complete Review Apps for CE and EE with GitLab QA running => 80%, Completed provisioning infrastructure, remaining polish work captured in Improving Review App Reliability
 - Complete phase 1 & 2 of Addressing database discrepancy for our on-prem customers => 70%, All schema discrepancy identified and documented. We still need to get more data on customer instances but have addressed all issues around missing index or extra index.
 - Support
 - Amer East: Develop at least 2 Premium service process improvements focused on improved coordination with TAMS, Premium customer ticket reduction efforts, and create premium ticket reports that give us more clarity of needs within our Premium customer base.
 - Amer West: Create a Support Engineer - GitLab.com- Bootcamp more tightly aligned with Production Engineering. Initial version completed in coordination with the production team: MR1, MR2
 - APAC: TBD process improvement (e.g. SLA performance, Customer Experience, or Employee Development)
 - EMEA: Enable support team to create 30 updates to docs to help capture knowledge and improve customer self service. Track contributions and views to measure 'self service score' (support edited doc visitors / customers opening tickets). Research completed. Tracking not yet

implemented

- UX:

- Create a seamless cluster experience. Identify 3 additional ways users can create and manage their clusters. Draft a proposal for an MVC that considers product, technical, and user requirements. Push for scheduling and implementation of the first iteration of group and instance clusters in Q4. Total: 83%. Efforts tracked in: Group level clusters product discovery: 16.67%, Instance level clusters product discovery: 16.67%, Serverless pathway discovery: 16.67%,

Group level clusters implemented: 16.67%, Instance level clusters implementation: 0% and Serverless tab implemented: 16.67%.

- Complete a Competitive analysis of version control tools for designers (Abstract, Invision, Figma, etc.). Identify ways GitLab can leverage Open Source and our own tools to assist designers using GitLab.. Total: 100%. Completed competitor analysis

- UX Research

- Establish a GitLab beta group and create a standardized process for running beta studies with users. Conduct 1 beta study with users to test and iterate on the process. Total: 90%.

- Security:

- Deprecate support for TLS 1.0 and 1.1 on GitLab.com: 100% Blog post

- Complete and document 12 month roadmap for ZTN rollout: 100% Kanban board

- Security Operations:

- Document at least 5 runbooks for logging sources: 5/5, 100% Internal repository

- Security and Priority labels on 50 backlogged security issues: 50/50, 100% List of issues

- Application Security:

- Complete at least 2 cycles of security release process: 2/2, 100% Releases

- Conduct at least 3 application security reviews: 3/3, 100% Kanban board

- Public HackerOne bounty program by December 15: 100% Blog post

- Compliance:

- Document current procedures for each of the 14 ISO Operations Security Domain: 100% Kanban board

- Create and publish two access templates 100% Access-request Project with access templates

- Conduct gap analysis of at least one GitLab.com subnet's access controls: 100% Baseline

Entitlements

- Abuse:

- Port existing 'janitor' project to Rails w/ 100% feature parity: 8/9, 89% Internal repository

- Support at least 2 external requests per month: 6/6, 100% Abuse tracker issue

- Product:

- Significantly improve navigation of the documentation. Ship global navigation. Improve version navigation. Revamp first page of the EE docs. => Total - 90% Complete- - Ship Global Nav - 100%, Improve version navigation - 70%, Revamp First Page - 100%

- Move vision forward every release. Prioritize at least one vision issue for each stage in every release. => 61% Complete. Of nine stages - 4 complete, 3 partials and two zeros.

- Every new feature ships with usage metrics on day 1, without negatively impacting velocity. => Missed ~0%. Significant majority of features shipped without usage metrics on day 1.

- CMO:

- Corporate marketing: Help our customers evangelize GitLab. 3 customer or user talk submissions accepted for DevOps related events.

- Corporate marketing: Drive brand consistency across all events. Company-level messaging and positioning incorporated into pre-event training, company-level messaging and positioning

reflected in all event collateral and signage.

- Corporate marketing: Increase share of voice. 20% lift in web and twitter traffic during event activity.

<p class="h3">CEO: Great team. ELO score per interviewer, executive dashboards for all key results, train director group.</p>

- CMO: Hire missing directors. Operations, Corporate, maybe Field.

- VPE: Craft an impactful iteration to improve our career development framework and train the team => 70%, shipped a new BE eng benchmark and experience factor worksheets (also for SREs) to hopefully address career development and comp for a meaningful proportion of the engineering function.

- VPE: Source 20 candidates by Oct 31 and hire 2 Engineering Directors: 100% (20/20) sourced, hired 1/2 directors => 60%, completed sourcing and hired a Sr Dir.

- Frontend:

- Scale process to handle incoming amount of applications and hire 2 engineering managers and 10 engineers: 1 engineering manager and 10 engineers hired (92%)

- Dev Backend:

- Gitaly:

- Source 10 candidates by October 15 and hire 1 developer: 10 sourced (100%), 2 hired (200%) => Hired one extra developer to compensate for Alejandro's departure

- Gitter:

- Create hiring process for fullstack developer => 100%

- Created hiring process and we are looking at candidates now,

<https://gitlab.com/gitlab-com/people-ops/hiring-processes/mergerequests/96>

- Plan:

- Source 25 candidates (at least 5 by direct manager contact) by October 15 and hire 2 developers: 25 sourced (100%), 2 hired (100%)

- Create:

- Source 25 candidates (at least 5 by direct manager contact) by October 15 and hire 2 developers: 25 sourced (100%), 2 hired (100%)

- Get 10 MRs merged into Go projects by team members without pre-existing Go experience: 6 merged, 1 in review (65%)

- Manage:

- Source 25 candidates (at least 5 by direct manager contact) by October 15 and hire 2 developers: 25 sourced (100%), 1 hired (50%)

- Team to deliver 10 topic-specific knowledge sharing sessions ("201s") by the end of Q4 (4 completed)

- Geo:

- At least one Geo team member to advance to Maintainer (0%) : Two engineers are in the group being trained to be maintainers.

- Distribution:

- Source 25 candidates (at least 5 by direct manager contact) by October 15 and hire 2 developers: 25 sourced (100%), 1 hired (50%)

- Stan:

- Reduce baseline memory usage in Rails by 30% => 50%

- Reduce runtime memory usage in Sidekiq for top 5 workers by 30% => 60%

- Resolve 3 P1 Tier 1 customer issues => 80%

- Infrastructure: Source 10 candidates by Oct 15 and hire 1 SRE (APAC) manager: X sourced

(X%) X hired (X%)

- SAE: Source 10 candidates by Oct 15 and hire 1 DBRE: 5 sourced (50%) 2 hired (200%)
- SRE: Source 25 candidates by Oct 15 and hire 2 SREs: 0 sourced (0%) 2 hired (100%)
- Ops Backend:
 - Hire 2 engineering managers (1 manager for Release, 1 manager for Secure): Hired 1 (50%)
 - Configure: Complete 10 introductory calls with prospective candidates by Nov 15 and hire 3 developers: 6 prospects (60%), hired 3 (150%)
 - Monitor: Source 30 candidates (at least 5 by direct manager contact) by November 15 and hire 3 developers: >30 sourced (100%), hired 2 (67%)
 - Secure: Source 75 candidates and hire 5 developers: 215 sourced (100%), 3 hired (60%)
 - CI/CD: Complete 10 introductory calls with prospective candidates by Nov 15 and hire 2 developers: 5 prospects (50%), hired 1 (50%)
- Quality:
 - Source 100 candidates (at least 5 by direct manager contact) by October 15 and hire 2 test automation engineers: 52 sourced (52%), 2 hired (100%)
- Security:
 - Source 100 candidates (at least 5 by direct manager contact) by October 15 and hire 3 security engineers: 100 sourced (100%), 6 hired (100%)
- Support: Source 30 candidates by October 15 and hire 1 APAC Manager 100% - hired new manager in November
 - Amer East: Source 30 candidates by October 15 and hire 1 Support Agent
 - Amer West: Source 30 candidates by October 15 and hire 1 Support Engineer: hired 1 Support Engineer (100%)
 - APAC: Source 30 candidates by October 15 and hire 1 Support Engineer
- UX:
 - Source 50 candidates by October 15 (at least 10 by direct manager contact) and hire 2 UX Designers and 1 UX Manager: 70 sourced (100%), 2 hired (66.6%)
- VP Alliances: Scale team to interact with our key partners. Hire Content Manager and Alliance Manager
- CFO: Improve processes and compliance
 - Legal: Implement a new contracting process to roll out with the vendor management process.
 - Legal: Update global stock option program to ensure global compliance.
- CFO: Create plan for preparing the Company for IPO
- CFO: Improve financial performance
 - FinOps: Annual planning process complete with company wide benchmarks and 100% of functions modeled to enable scenario analysis.
 - FinOps: Headcount planning linked to requisition process that provides for recruiting and hiring manager visibility against plan assumptions.
- CCO: Improve Recruiting and People Ops Metrics, including relevant benchmarking data
 - Recruiting: Hiring and Diversity stats and goals defined by functional group
 - Recruiting: Rent Index Evaluation by functional group
 - Recruiting: Single Source of truth for Recruiting data, ownership and progress. If we can get this down to 2 sources in Q4, that will be a win.
- PeopleOps: Added clarity to the turnover data and industry benchmarking. Attempt to get benchmark data for remote companies.
- CCO: Continue on the next iteration of Compensation
 - PeopleOps: Review and revise benchmarks to meet our current Compensation Philosophy
 - PeopleOps: Explore and decide on revising the compensation philosophy that supports hiring and retaining top talent, and have the plan in place to move forward.

- PeopleOps: Review and revise Rent Indexes to align with the compensation philosophy.
- CCO: Continue to improve Career Development and Feedback
- Redesign the experience factor model to focus on supporting development and conduct trainings.
- Define Cross-Functional and Functional Group specific VP and Director Criteria
- Launch an employee engagement survey that results in at least 85% participation and 3 themes to work on as a leadership team.
- CCO: Improve Recruiting efficiency and effectiveness to build a solid foundation to scale:
 - Clearly define and standardize process, roles, responsibilities, and SLA's
 - Improve continuity and accountability through recruiter/sourcer/coordinator alignment
 - Launch candidate survey in Greenhouse
 - Implement and integrate new background check provider
 - Enhance candidate communication and templates (invites, updates, decline notices)
 - Optimize Greenhouse and provide refresher training to hiring managers
 - Support Headcount planning process driven by Finance to ensure one source of truth and process for hiring plan and recruiter capacity
- Continue work on ELO score for interviews. First iteration complete by mid November.
- Product: Hire 2 Directors of Product => 100% Complete- - hired Kenny and Eric

Engineering (VPE)

- GOOD
 - Shipped several new impactful salary benchmarks
 - Shipped meaningful career development asset for BE engineers in the form of experience factor worksheets
 - GitLab.com is now meeting business goals
 - Error budgets allowed us to manage risk and solve some problems
- BAD
 - Did not hire the second director of engineering we planned to
 - Did not address comp for every engineering role
 - Did not address career development for every engineering role
- TRY
 - Delegate OKR process for managers to my directors in Q1
 - End error budget experiment in OKRs
 - Only do 2019 Q1 hiring KRs for teams that are not regularly hiring to plan
 - Delegate Director of Engineering hires to Sr Director of Development in Q1
 - Address comp expectations and career development for the remaining roles in engineering in Q1
 - Assume .com continues to meet business goals and not take on the mission critical goal in favor of more granular infra KRs in Q1
 - Arrange training in Q1 for career development facets (like writing secure code) for engineers

Frontend

- GOOD
 - Met our engineering hiring goal and very close to closing also second manager. Ramped up the team for Hiring fast and the whole pipeline is now a team effort
 - Able to scale gitlab-ui, have the full workflow and the SSOT with design.gitlab.com while most of the work on documentation is automated

- Onboarding new members with the frontend buddy system and constant prioritization of new hires worked well
- We had a good analytical process of selecting a future proof charting library and drove it to integration a release cycle later
- BAD
 - Scaling our CSS efforts is not effective enough and especially doesn't scale
 - Work on performance and technical base work slowed down
 - Our assets grow sometimes suddenly due to changes and without looking constantly at Webpack reports no one is aware of it
- TRY
 - Replan, Restructure and Rethink our CSS refactoring
 - Focus on performance improvements and distribute that wider in the team
 - Even as our tooling improved, there is way more that we can do to improve workflows, automation, setup time and consistency by tailoring solutions 100% to our needs
 - Bring more tooling for JS development to GitLab itself so we have it integrated in our workflow but also others can profit (Webpack reports, etc.)

Dev Backend

- GOOD
 - Changes to retrospectives seem well-received: feedback is that they feel more productive and flow more cleanly
 - Clarified some terminology confusion with Product around prioritization
- BAD
 - Cross-functional collaboration was hard to push due to absences.
 - Hard to have teams prioritize as we'd like due to gaps in the product.
 - Had a 'regression' in retrospective due to changing several things at once
- TRY
 - Pushing more aggressively to prioritize what we need in the product
 - Keep in mind cross-functional collaboration may be difficult in Q4 due to holidays
 - Keep policy iterations (e.g. changes to retrospective format) atomic to better track success and avoid regressions

Plan

- GOOD
 - Met the hiring target
 - No production incidents
 - One more person trained to lead technical interviews, another close to finishing their training
 - Single prioritised list works well for the team
- BAD
 - Despite a significant amount of effort towards the Elasticsearch goal, we have not made any significant reductions in index disk usage
 - Elasticsearch issues have a tendency to get lost or deprioritised
 - One person left the team
 - Although we aren't the most-impacted team, security issues have taken a lot of the team's time over the past few months

- TRY
 - Reframing Elasticsearch goal as improving admin controls, reduction in downtime during upgrades, progressive enablement, etc. These issues allow more incremental progress and don't block improvement in index size
 - Reviewing security issues to ensure that we are clearing a backlog, rather than introducing new issues at the same rate as we fix old ones

Distribution

- GOOD
 - The whole team is participating in all team projects (both Helm Charts and omnibus-gitlab)
 - Started with team trainings again which helped the team share knowledge faster
 - Keeping up with the incoming issues, all projects with open issues are trending down
 - Exposure of Helm Charts is growing and with that the feedback received
- BAD
 - With the expanded work scope, context switching spiked
 - Helm Charts are taking majority of the teams time, which impacts the omnibus-gitlab package feature development
 - Higher complexity of the Helm Charts and the relative immaturity of tooling is making the team less efficient
 - Due to the new nature of Helm Charts, documentation is still lacking in spite of teams best effort to keep it updated frequently causing confusion with users
- TRY
 - Find a way to remove duplicate work between the omnibus-gitlab package and Helm Charts in regards to introducing new configuration
 - Focus more on initial installation experience for the new users
 - Train more teams to be self sufficient with the Helm Charts
 - Ensure that the omnibus-gitlab package keeps receiving enough attention

Geo

- GOOD
 - Reinstated the monthly team call which was received well by the team.
 - Cycle time on reviews (done on Geo work by the Geo team) has been reduced by concentrating on making sure that others are unblocked.
 - The team felt that they accomplished a lot on items like bug fixing, refactoring, performance improvements and documentation.
- BAD
 - Some issues were not as prepared as they could have been and as a result the issues included in the milestone changed a lot during a month.
 - For some issues we were delayed in waiting for specialized reviewers. We know that there are improvements to the DB review process coming soon, and we look forward to these changes being in place.
 - We struggled with an erroneous revert as part of the automatic CE->EE merging. It wasn't immediately clear that the revert had happened.
 - It was hard to know if some of the QA failures were due to our work or due to work on the QA framework itself. It is difficult and slow to figure out the difference
- TRY

- We need to spend more time in planning. We will introduce a process in 11.8 and see how this enables us to have more predictable deliverables in a milestone

Manage

- GOOD

- Stayed within the error budget, something we were nervous about after having an incident so early in the quarter.

- We kept focus on deliverables for much of the quarter (e.g SmartCard, Group SAML, Billing improvements), which feels like an achievement given the size and scope of the team.

- A decision has been made to create a new team ('Sync'), which will help bring more focus to areas such as Billing & Licensing, while reducing the scope of the Manage team.

- Made a conscious decision to prioritize reducing the security backlog in the coming milestones.

- The impact of the 201 sessions (now known as Deep Dives) was really positive and has encouraged other teams to do similar.

- BAD

- Only hired 1 out of 2 positions.

- Prioritization of security Issues will impact Deliverables until we clear down the backlog.

- Number of Issues delivered was inconsistent for the 3 milestones in the quarter, making it hard to predict future delivery.

- TRY

- Work closer with the recruitment team, and other engineering teams, to understand what we need to do to speed up hiring.

- Experiment with milestone planning to increase both velocity and predictability.

Create

- GOOD

- Met the hiring target!

- No error budget points used

- 2 people without significant pre-existing Golang experience are now comfortable contributing to our Golang projects

- The pressure on and around the feature freeze was reduced by the new policy that allows MRs to be picked into stable after the 7th as long as they have a feature flag.

- BAD

- 1 team member left

- 3 out of 5 people without significant pre-existing Golang experience did not end up contributing to Golang projects

- We continue to overestimate the amount of work we can get done in a month, through underestimating the size of individual issues, resulting in higher pressure around the feature freeze and more slipping to the next release than desirable

- Security issues have taken up a significant portion of each of the last few releases, resulting in less time to work on direction features

- TRY

- Continue to have people without Golang experience work on issues that require them to get familiar with our Golang components (Gitaly, Workhorse)

- Break issues down more before scheduling them so that we have a better idea of the amount of work required and we can release them one part at a time, instead of the whole big MR slipping multiple releases in a row

Gitaly

- GOOD
 - Delivered on 100% of OKRs
 - Hiring finally picked up for team
- BAD
 - Delivered on 100% of OKRs :) Goals probably weren't aggressive enough.
- TRY
 - Setting more aggressive goals, particularly around delivering more rapid architectural improvements

Gitter

- GOOD
 - Gitter is remaining resilient and stable
 - Hiring process created and we are looking at candidates now
 - First version from open source repo of the Gitter Android app published to Google Play Store via GitLab CI, <https://gitlab.com/gitlab-org/gitter/gitter-android-app/mergerequests/5>
- BAD
 - Infrastructure is a bit outdated and in need of love. Someone was able to find an exploit and run a malicious process on a couple of our servers
- TRY
 - Update and refresh infrastructure, <https://gitlab.com/gitlab-com/gl-infra/infrastructure/issues/5492>
 - Keep up with Security reports from HackerOne

Stan

- GOOD
 - Identified and fix an NFS bug for a number of customers
 - Reduced ~80 MB baseline memory usage by eliminating Linguist dependency
 - Reduced memory usage of the top 2 worst performing Sidekiq jobs by several gigabytes (UpdateMergeRequestsWorker and ReactiveCachingWorker)
 - Formalized maintainer role and started reviewing much more code
- BAD
 - OKRs didn't reflect a significant amount of the day-to-day work
 - Got pulled into scoping and improving audit logging for customer
 - Implemented Bitbucket Server importer for a customer
 - Got sidetracked helping many other teams (Geo, Infrastructure, etc.) and fixing production incidents (e.g. <https://gitlab.com/gitlab-org/gitlab-ce/issues/53153>)
- TRY
 - Making OKRs actually reflect work done

Infrastructure

- GOOD
 - We did well with hiring and onboarding
 - We improved our ability to manage workload (even as it still needs more work)
 - We cleaned up a significant backlog of team issues, which will allow us to fully focus on executing

- BAD

- OKRs were iffy at best: we treated them like homework rather than a proper management tool
- We underestimated our workflow management at scale, which created confusion in the team
- We failed to communicate new error budget framework widely

- TRY

- Finish adopting OKRs and develop the framework of OKRs through GitLab issues
- Ensure the OKR -> Epic -> Milestone -> Issue is crystal clear

SAE

- GOOD

- Hired 3 DBRE's.
- Delivered a good percentage of the OKR's and another expressive project that was Patroni.

- BAD

- We missed an OKR to represent the project of Patroni.

- TRY

- Improve production best practices, work on better documentation and processes, to avoid self-inflicted outages.

SRE

- GOOD

- Got measurements in team velocity spreadsheet
- Continuing to bring new team members up to speed
- Hired 2 more SREs 1 EMEA / 1 Americas

- BAD

- DR not fully completed in quarter
- 1 queue of issues for all team created information/tracking overload

- TRY

- Breaking apart team milestones to smaller groups tied to team level OKRs
- Small sprint to finish DR work

Andrew

- GOOD

- Slack alerts to service issues with embedded grafana graphs to help visualise problems and reduce MTTD.

- Delivered Correlation IDs. These are already proving very useful in diagnosing production problems and also in self managed instances. This also formed the basis for distributed tracing, which will be delivered soon.

- Abuse detection alerting is being used by the abuse team to detect and stop abuse.

- BAD

- Delivered two large, unplanned work pieces: Puma/Multithreaded App Server and Google Capacity Planning which were not accounted for in OKR planning or scoring.

- More could be done to train team members on how to use new tools

- TRY

- Adjust OKRs during the quarter if priorities change
- Focus on emitting more documentation through blueprints, videos and user documentation.

Ops Backend

- GOOD
 - Completed goals for throughput OKR thanks to the support of the team to try a new model and seeing success with it.
 - Hired an engineering manager for release. Very happy to welcome Darby Frey to GitLab!
- BAD
 - Did not hire a manager for secure. This has been a tough role to hire for despite our increased efforts in sourcing and filling the pipeline. We are seeing an uptick in the number of interviews in Q1 likely due to the efforts from Q4 but it's disappointing that we are still in search.
- TRY
 - Work with recruiting to diversify and improve our sourcing for the Secure team manager role.
 - Improve our presence at events and help widen our brand awareness on LinkedIn and other recruiting sites

Monitoring

- GOOD
 - Sourced more candidates than our goal
 - No error budget points spent
- BAD
 - Only hired 2 out of 3 positions
 - Didn't meet throughput goal (only 79%)
 - Didn't spend any error budget points. This is mostly due to our surface area still being so small.
- TRY
 - Encourage smaller MRs merged earlier. Right now we have a number of small MRs, but they are all getting merged late in the cycle.

CI/CD

- GOOD
 - Average throughput was significantly higher than our goal
- BAD
 - Spent double our goal error budget
 - Only hired one engineer
- TRY
 - More active sourcing and referral seeking to fill roles
 - Add some structure to how we prioritise and review community MRs to increase throughput

Secure

- GOOD
 - Sourced more candidates than expected (215)
 - No error budget points spent
 - Met throughput goal
- BAD
 - Only hired 3 out of 5 positions
- TRY
 - Regarding our rejection ratio, source even more candidates

Configure

- GOOD
 - Hired more than target, thanks to referrals and introductory calls
- BAD
 - Didn't meet the throughput goal (only 87%)
 - Reluctance to split GitLab issues due to the challenge of splitting our communication across multiple issues (significant tradeoff with the extensive async communication)
 - Not a single hire came from an application (all referral and sourced for introductory calls). Not clear if this is a problem since we exceeded our hiring target.
- TRY
 - Stress the importance of smaller MRs more also try to make smaller issues
 - Ask for referrals more, encourage team members to share job ads on LinkedIn
 - Continue our focus on Introductory calls since 6 calls led to 1 hire which is a good return on investment

Kamil

- GOOD
 - Managed to finish major improvements related to CI scalability: database footprint and fixing Object Storage bugs,
- BAD
 - The amount of OKR tasks didn't really fit into spare time. I was constantly struggling with trying to find time for OKRs while doing regular development / reviews.
- TRY
 - Focus on smaller tasks or define explicitly that OKR is not done in spare time, but rather in 1/3 of time to have better focus on.
 - Stop (disconnect) other interruptions while working on OKRs.

Philippe

- GOOD
 - Several CFPs were posted, I should have at least one accepted.
- BAD
 - Didn't have time to benchmark any tool. It requires to be focused and available, which I'm not because of my interim EM position.
- TRY
 - Find a new Secure Engineering Manager to allocate time for my new role OKRs.

Quality

- GOOD
 - Hired two test automation engineers.
 - Completed Patroni Replication Manager testing with GitLab QA.
 - Completed TLS Depreciation testing with GitLab QA.
 - Completed Rails 5 migration testing with GitLab QA.
- BAD
 - An update to the clone / fork button cause GitLab QA tests to break for 4 days.

- Review Apps are broken (no DNS record) due to an increasing number of DNS records and too many API calls to AWS.
- We had a few high severity regressions were as a result of Rails 5.
- We are still lagging behind on test gaps.
- TRY
 - We need to ensure that broken QA tests has the same urgency as a broken master.
 - Incentivize and encourage engineers to write E2E tests.

Security

- GOOD
 - Public HackerOne bug bounty program rollout in December, with 1 hour first response time to new reports.
 - Security on-call rotation and automation successfully deployed with SLA of 15 minutes.
 - Completed initiative of discontinued support of TLS 1.0 and 1.1 on GitLab.com.
 - MTTM for S1/P1 security vulnerabilities consistently meets 30 day timeframe.
 - Biweekly application security office hours have been helpful so far.
- BAD
 - Security releases, both critical and non-critical continue to be a challenge, with regression issues for previously hotpatched issues on GitLab.com.
 - MTTM of S2/P2 security issues are not consistently meeting 60 day timeframe.
- TRY
 - Work with release delivery and management team to address security release issues.
 - Deploy mandatory onboarding secure coding training for all new (and existing) developers, to lower overall vulnerable code.

Support

- GOOD
 - Positive team reaction to workflow changes to improve SLA response. Impressive KPI results with FRT trending close to 95% by EOQ.
 - Good cross-team coordination with Recruiting and hiring to plan.
 - Completed upgrade to Zendesk Enterprise and initial update to Zendesk workflows.
- BAD
 - Experience factor review process had unnecessary friction suggesting iteration for a smoother Compensation process.
 - Although hiring goals were met we need to partner with Recruiting to lessen time to hire new roles.
- TRY
 - More effectively surface common product challenges from customers into Product.
 - Smoother customer on-boarding with Sales (<https://gitlab.com/gitlab-com/support/support-team-meta/issues/1323>)

Americas East

- GOOD
 - First Response Hawk Process is improving SLAs
 - Team adopted more async workflows (Issue boards/Week-in-review doc)
 - Offboarded APAC Support team to new manager successfully.
 - Will C. Developed Strace-parser which has helped tremendously with customer performance

debugging.

- BAD

- We need more Geo Expertise.

- Support Fix and Documentation processes were not as clearly defined as they should be which lead to delays and friction in using them.

- TRY

- Better communication and collaboration with docs to get changes merged in faster, more accurately.

- Develop FRT Hawk Tooling to improve efficiency

- Get other regions to use strace-parser through better training and documentation.

Americas West

- GOOD

- Hired to target

- Have a solid first iteration on .com Engineering onboarding and clear next steps

- .com performance in December was exceptional (after our BAD day)

- Adhoc pairing sessions have reduced breaches and increased team knowledge

- BAD

- Weekend merge of a 4-hr SLA on Premium left the .com team scrambling

- Coverage gap left .com with a 0% SLA day in December (only 1 ticket actually breached)

- TRY

- Have .com team members play the role of CMOC in the incident management process

- Rotate through "FRT Hawk" role throughout team

APAC

- GOOD

- Manager hired for the region

- BAD

- Length of time to recruit manager for APAC

- TRY

- Define regional hours of coverage with the team and cover appropriately

- Participate in the "FRT Hawk" role once completed in Americas

- Define framework for multi-language support

EMEA

- GOOD

- Hired 2 Support Agents per target

- Focused learning objectives for team with quarterly learning goals

- Groundwork for knowledge sharing, documentation updates, search and tracking

- Streamline Zendesk configuration to improve agent efficiency

- BAD

- Tracking of knowledge capture and documentation updates not yet in place

- Frontend search of knowledge to deflect support tickets not yet in place

- TRY

- Track documentation updates and knowledge sharing by Support

- Implement search of knowledge to deflect support tickets and track effectiveness

- Continue to streamline and improve Zendesk configuration with help of Support Operations specialist when hired.

UX

- GOOD

- We developed and finalized an MVC for both group and instance level clusters with collaboration between UX, Frontend, Backend, and Product.
- We also created another user journey towards creating a project cluster by introducing the Serverless tab. This navigates users towards the cluster page if they do not already have one. This was a joint effort between the Configure team and Triggernesh (a contracted group).
- Completed a comprehensive competitor analysis of version control tools for designers.
- Hired 2 UX Designers.

- BAD

- While we pushed for group and instance level clusters to be implemented, instance level clusters did not make it in Q4. The team did not have capacity.
- Didn't hire a UX Manager.

- TRY

- Better defining what we want and need from UX Managers.

Research

- GOOD

- We launched GitLab First Look, a new comprehensive research program for users of GitLab. Users are now able to sign-up to beta testing with GitLab.
- Great collaboration with Product and Marketing.

- BAD

- Due to the holidays, some of the features we planned to beta test missed RCs/11.6 release therefore we've not had the opportunity to test out and re-iterate on our beta testing process.

- TRY

- Test the beta testing process for features in 11.7

SECTION: company/okrs/_index.md

This page generally covers OKRs at GitLab, including:

1. Considerations and guidance
1. OKR Process at GitLab
1. Maintaining the health status of OKRs
1. Scoring OKRs

There is additional information on:

1. What OKRs are, and general guidance on how to formulate them, on the general OKRs page.
1. How to enter and organize OKRs in GitLab, on the OKRs in GitLab page.

Overview

OKRs are quarterly objectives that help us achieve our KPIs.

We do not use it to give performance feedback or as a compensation review for team members.

The E-Group does use it for their Performance Enablement Reviews.

The Chief of Staff to the CEO initiates and guides the OKR process.

Most recent OKRs

Our OKR process and timelines are public and listed on the pages below.

- FY25-Q4 Active

OKRs are internal-only in line with guidance from the SAFE framework.

OKRs are what is different

The OKRs are what we are focusing on this quarter specifically. Our most important work are things that happen every quarter.

Things that happen every quarter are measured with Key Performance Indicators.

Part of the OKRs will be or cause changes in KPIs.

Measuring Brand New Initiatives

Some KRs will measure new approaches or processes in a quarter. When this happens, it can be difficult to determine what is ambitious and achievable because we lack experience with this kind of measurement. For these first iterations, we prefer to set goals that seem ambitious and expect a normal distribution of high, medium, and low achievement across teams with this KR.

Shared Objectives

If there is something important that requires two (or more) parts of our organization, all leaders involved should share the same or similar objective. They should have deconflicted key results so they can still achieve things within their sphere of control. This is in keeping with our concepts of collaboration and directly responsible individual (DRI).

Pass-thru Key Results

It's acceptable for managers and reports to have an identical key result. For instance, something really important might need to happen at the executive level, but it's a manager or IC several layers apart who is doing the actual execution. Every person in that line of reporting should have the same key result.

While it can feel like double-counting, it is consistent with Andy Grove's concept of Managerial Leverage outlined in his book High Output Management. This ensures that conversations happen in the relevant 1-1s, that everyone knows the latest status, and that the person executing does not accidentally get re-tasked. Please remember to recognize the person that achieved the result so there is no perception of "taking credit" for others' work.

Target Dates in Key Results

Just because quarters are a useful timeframe for objectives, does not mean that key results should default to being due on the last day of the quarter. This could lead to unnecessary delays. Consider putting specific target dates into the key result phrase to indicate when it is needed by.

You should decide your scoring methodology ahead of time. You might score an OKR as 0% if it misses its target date. Or, if it's less time sensitive, you might penalize it 10% for each week it's delayed.

How do I prioritize OKRs in light of other priorities?

OKRs do not replace or supersede core team member responsibilities or performance indicators. OKRs are additive and are essentially a high signal request from your leadership team to prioritize the work. They typically are used to help galvanize the entire company or a set of teams to rapidly move in one direction together. You should aim to complete them unless you have higher priority work that is surfaced and agreed to by leadership. When surfacing tradeoffs, team members should not factor in how an unmet OKR may impact your executive leadership bonus in their prioritization. They should instead focus on GitLab priorities. If your executive leader still feels that the OKR is more important, they will ask you to disagree and commit.

OKR Process at GitLab

The CoS to the CEO is coordinating the OKRs process detailed below. The EBA to the CEO assists in scheduling designated time during E-Group Weekly Meetings.

CEO initiates new quarter's OKRs

In the first month of the fiscal quarter, the Chief of Staff (CoS) to the CEO initiates the OKR process. The CoS works with the CEO will propose big themes and priorities for next quarter. Yearlies are consulted in the drafting process. The CoS to the CEO creates a Google Doc for E-Group alignment. This document is shared with E-Group in an E-Group Weekly. E-Group is encouraged to offer feedback in the E-Group Weekly, directly within the Google Doc, or in meetings with the CEO or Office of the CEO.

In the second month of the fiscal quarter, E-Group will align on big themes and priorities. Before the end of the second month and before team planning is deeply underway, E-Group will lock in a single set of company-wide OKRs. After E-Group alignment, Company-level OKRs will then be shared with all of GitLab in the okrs cha